CPU Scheduling Algorithms Simulator

CSCI 4103 Course Project

Maggie Hawkins

2/17/2026

INTRODUCTION

The objective of this course project was to implement and simulate CPU scheduling algorithms. I was asked to implement first-come, first serve (FCFS), Shortest Process Next (SPN), Round Robin (RR), and Priority Scheduling (Non-Preemptive). I chose to implement these algorithms using Python. Upon review of my simulation, I was able to ask users to input process details and then they received calculations on performance metrics. I also created a function that allowed Gantt charts to be displayed. Lastly, I used a few test runs of data to ensure my algorithms worked.

PROCEDURE

1. User Inputs Process Details
2. Averages are Calculated
3. Gantt Charts are Displayed

EXPLANATION

Let's break down how each algorithm works.

First-Come, First-Serve (FCFS)
FCFS schedules processes in the order they arrive. It is non-preemptive like the Priority Scheduling algorithm, meaning once the process starts, it runs until complete. I found this algorithm to be simple but it has its drawbacks. It can cause longer wait times for shorter processes because they are executed in order.

Shortest Process Next (SPN)
SPN is another non-preemptive algorithm. Unlike FCFS it chooses order based on processes with the smallest burst time that have already arrived. Overall, the average wait time is less than FCFS but longer processes can lead to starvation.

Shortest Remaining Time (SRT)
SRT is the first preemptive algorithm we handled. It's like SPN in the fact that it chooses based on the smallest remaining burst time, but it's preemptive. This means it is constantly choosing the newest process with the smallest burst time, preempting the other process.

Round Robin (RR)
Round Robin algorithms are also preemptive. Each process is assigned a time quantum, in our data results below you will see I assigned 2. If the process doesn't finish within the set quantum, it is placed back in a queue while the next process is selected.

Priority Scheduling (PS)
The last algorithm we created in our project is priority scheduling. It's a non-preemptive algorithm that selects processes based on priority level. It can be effective in task management but can starve lower priority processes.

DATA RESULTS

Test cases and Results

For every algorithm I used the same numbers to ensure each program works appropriately. I enter each number in the following order.

Enter number of processes: 4
****** Then the Process Number would follow automatically******
Process 1
Arrival Time 1
Burst Time 2
Priority 3
Process 2
Arrival Time 4
Burst Time 5
Priority 6
Process 3
Arrival Time 7
Burst Time 8
Priority 9
Process 4
Arrival Time 4
Burst Time 6
Priority 8

It would return the waiting time, turnaround time, and response time for each process.

Next, it would calculate the averages:

Average Waiting Time 3.75
Average Turnaround Time 9.0
Average Response Time 3.75

Followed by displaying a Gantt chart.

All 5 algorithms work and return the appropriate averages and charts.

The last averages were for FCFS. Let me show the averages and gantt chart for each algorithm using the same numbers I used above!

SPN:
Average Waiting Time 3.25
Average Turnaround Time 8.5
Average Response Time 3.25
Gantt Chart:
| P1 (1-3) | P2 (4-9) | P4 (9-15) | P3 (15-23)

SRT:
Average Waiting Time 3.25
Average Turnaround Time 8.5
Average Response Time 3.25
Gantt Chart:
| P1 (1-2) | P1 (2-3) | P2 (4-5) | P2 (5-6) | P2 (6-7) | P2 (7-8) | P2 (8-9) | P4 (9-10) | P4 (10-11) | P4 (11-12) | P4 (12-13) | P4 (13-14) | P4 (14-15) | P3 (15-16) | P3 (16-17) | P3 (17-18) | P3 (18-19) | P3 (19-20) | P3 (20-21) | P3 (21-22) | P3 (22-23)

RR: (With a quantum of 2 entered)
Average Waiting Time 5.75
Average Turnaround Time 11.0
Average Response Time 1.25
Gantt Chart:
| P1 (1-3) | P2 (4-6) | P4 (6-8) | P2 (8-10) | P3 (10-12) | P4 (12-14) | P2 (14-15) | P3 (15-17) | P4 (17-19) | P3 (19-21) | P3 (21-23)

PS:
Average Waiting Time 3.25
Average Turnaround Time 8.5
Average Response Time 3.25
Gantt Chart:
| P1 (1-3) | P2 (4-9) | P4 (9-15) | P3 (15-23)

CONCLUSION

The non-preemptive algorithms felt simpler in construction whereas the preemptive ones took a little more time. Overall, I felt that this project helped me practice my coding while implementing material we've learned in class. I've spent lots of time inputting different data into the program and am fascinated at how each algorithm works differently.

REFERENCES

1. *Computer Coding Picture*. (n.d.). https://www.computersciencedegreehub.com/faq/what-is-coding/. Retrieved from https://www.computersciencedegreehub.com/faq/what-is-coding/.

MY PROGRAM: Juptyer Notebook

```
def fcfs(processes):
    processes.sort()
    time = 0
    results = {}
    timeline = []

    for p in processes:
        pid, arrival, burst, priority = p
        if time < arrival:
            time = arrival
        start = time
        end = time + burst
```

```
        print("Running processes with burst", burst, "from", time, "to", end)
        timeline.append((pid, start, end))


        results[pid] = (arrival, burst, start, end)

        time = end

    total_wait = 0
    total_turn = 0
    total_resp = 0

    for pid in results:
        arrival, burst, start, finish = results[pid]

        turnaround = finish - arrival
        response = start - arrival
        waiting = start - arrival

        total_wait += waiting
        total_turn += turnaround
        total_resp += response


        print("Process", pid, "Waiting Time:", waiting)
        print("Process", pid, "Turnaround Time:", turnaround)
        print("Process", pid, "Response Time:", response)

    avg_wait = total_wait / len(results)
    avg_turn = total_turn / len(results)
    avg_resp = total_resp / len(results)

    print("Average Waiting Time", avg_wait)
    print("Average Turnaround Time", avg_turn)
    print("Average Response Time", avg_resp)
```

```
    print("\nGantt Chart:")
    for pid, start, end in timeline:
        print(f"| P{pid} ({start}-{end}) ", end="")
    print()

def spn(processes):

    time = 0
    results = {}
    timeline = []

    remaining = processes.copy()

    while remaining:

        available = []

        for p in remaining:
            pid, arrival, burst, priority = p
            if arrival <= time:
                available.append(p)

        if not available:
            time = min(p[1] for p in remaining)
            continue

        chosen = available[0]

        for p in available:
            if p[2] < chosen[2]:
                chosen = p

        pid, arrival, burst, priority = chosen

        start = time
        end = time + burst
```

```
        print("Running process", pid, "from", start, "to", end)
        timeline.append((pid, start, end))


        results[pid] = (arrival, burst, start, end)

        time = end

        remaining.remove(chosen)

    total_wait = 0
    total_turn = 0
    total_resp = 0

    for pid in results:
        arrival, burst, start, finish = results[pid]

        turnaround = finish - arrival
        response = start - arrival
        waiting = start - arrival

        total_wait += waiting
        total_turn += turnaround
        total_resp += response


        print("Process", pid, "Waiting Time:", waiting)
        print("Process", pid, "Turnaround Time:", turnaround)
        print("Process", pid, "Response Time:", response)

    avg_wait = total_wait / len(results)
    avg_turn = total_turn / len(results)
    avg_resp = total_resp / len(results)

    print("Average Waiting Time", avg_wait)
    print("Average Turnaround Time", avg_turn)
    print("Average Response Time", avg_resp)
```

```
    print("\nGantt Chart:")
    for pid, start, end in timeline:
        print(f"| P{pid} ({start}-{end}) ", end="")
    print()

def srt(processes):

    time = 0
    results = {}
    timeline = []

    remaining = []

    for p in processes:
        pid, arrival, burst, priority = p
        remaining.append([pid, arrival, burst, burst])

    first_start = {}

    while remaining:

        available = []

        for p in remaining:
            pid, arrival, burst, remaining_time = p
            if arrival <= time:
                available.append(p)

        if not available:
            time += 1
            continue

        chosen = available[0]

        for p in available:
            if p[3] < chosen[3]:
```

```
            chosen = p

    pid, arrival, burst, remaining_time = chosen

    print("Running process", pid, "at time", time + 1)
    timeline.append((pid, time, time + 1))

    if pid not in first_start:
        first_start[pid] = time

    chosen[3] -= 1

    if chosen[3] == 0:
        finish_time = time + 1
        results[pid] = (arrival, burst, first_start[pid], finish_time)
        remaining.remove(chosen)

    time += 1

    total_wait = 0
total_turn = 0
total_resp = 0

for pid in results:
    arrival, burst, start, finish = results[pid]

    waiting = finish - arrival - burst
    turnaround = finish - arrival
    response = start - arrival


    total_wait += waiting
    total_turn += turnaround
    total_resp += response


    print("Process", pid, "Waiting Time:", waiting)
```

```
        print("Process", pid, "Turnaround Time:", turnaround)
        print("Process", pid, "Response Time:", response)

    avg_wait = total_wait / len(results)
    avg_turn = total_turn / len(results)
    avg_resp = total_resp / len(results)

    print("Average Waiting Time", avg_wait)
    print("Average Turnaround Time", avg_turn)
    print("Average Response Time", avg_resp)

    print("\nGantt Chart:")
    for pid, start, end in timeline:
        print(f"| P{pid} ({start}-{end}) ", end="")
    print()

def rr(processes):

    quantum = int(input("Enter time quantum:"))

    time = 0
    results = {}
    timeline = []
    first_start = {}

    queue = []
    remaining = []

    for p in processes:
        pid, arrival, burst, priority = p
        remaining.append([pid, arrival, burst, burst])

    while remaining or queue:

        for p in remaining[:]:
            pid, arrival, burst, remaining_time = p
            if arrival <= time:
```

```
        queue.append(p)
        remaining.remove(p)

total_wait = 0
total_turn = 0
total_resp = 0

if not queue:
    time += 1
    continue

current = queue.pop(0)

pid, arrival, burst, remaining_time = current

if pid not in first_start:
    first_start[pid] = time

run_time = min(quantum, remaining_time)

print("Running process", pid, "from", time, "to", time + run_time)
timeline.append((pid, time, time + run_time))

current[3] -= run_time

time += run_time

for p in remaining[:]:
    pid2, arrival2, burst2, remaining_time2 = p
    if arrival2 <= time:
        queue.append(p)
        remaining.remove(p)

if current[3] > 0:
    queue.append(current)
else:
    results[pid] = (arrival, burst, first_start[pid], time)
```

```
    for pid in results:
        arrival, burst, start, finish = results[pid]

        waiting = finish - arrival - burst
        turnaround = finish - arrival
        response = start - arrival


        total_wait += waiting
        total_turn += turnaround
        total_resp += response


        print("Process", pid, "Waiting Time:", waiting)
        print("Process", pid, "Turnaround Time:", turnaround)
        print("Process", pid, "Response Time:", response)

    avg_wait = total_wait / len(results)
    avg_turn = total_turn / len(results)
    avg_resp = total_resp / len(results)

    print("Average Waiting Time", avg_wait)
    print("Average Turnaround Time", avg_turn)
    print("Average Response Time", avg_resp)

    print("\nGantt Chart:")
    for pid, start, end in timeline:
        print(f"| P{pid} ({start}-{end}) ", end="")
    print()

def priority_sched(processes):

    time = 0
    results = {}
    timeline = []
```

```
remaining = processes.copy()

while remaining:

    available = []

    for p in remaining:
        pid, arrival, burst, priority = p
        if arrival <= time:
            available.append(p)

    if not available:
      time = min(p[1] for p in remaining)
      continue

    chosen = available[0]

    for p in available:
        if p[3] < chosen[3]:
            chosen = p

    pid, arrival, burst, priority = chosen

    start = time
    end = time + burst

    print("Running process", pid, "from", start, "to", end)
    timeline.append((pid, start, end))


    results[pid] = (arrival, burst, start, end)

    time = end

    remaining.remove(chosen)

total_wait = 0
```

```
    total_turn = 0
    total_resp = 0

    for pid in results:
        arrival, burst, start, finish = results[pid]

        turnaround = finish - arrival
        response = start - arrival
        waiting = start - arrival

        total_wait += waiting
        total_turn += turnaround
        total_resp += response



        print("Process", pid, "Waiting Time:", waiting)
        print("Process", pid, "Turnaround Time:", turnaround)
        print("Process", pid, "Response Time:", response)

    avg_wait = total_wait / len(results)
    avg_turn = total_turn / len(results)
    avg_resp = total_resp / len(results)

    print("Average Waiting Time", avg_wait)
    print("Average Turnaround Time", avg_turn)
    print("Average Response Time", avg_resp)

    print("\nGantt Chart:")
    for pid, start, end in timeline:
        print(f"| P{pid} ({start}-{end}) ", end="")
    print()


print("Program started")
n = int(input("Enter number of processes:"))
processes = []
for i in range(n):
```

```
    print("Process", i+1)
    arrival = int(input("Arrival Time:"))
    burst = int(input("Burst Time:"))
    priority = int(input("Priority"))
    processes.append((i+1,arrival, burst, priority))
print(processes)

print("Select Scheduling Algorithm:")
print("1. FCFS")
print("2. SPN")
print("3. SRT")
print("4. RR")
print("5. PS")
choice = int(input("Enter your choice:"))

if choice == 1:
    fcfs(processes)
if choice == 2:
    spn(processes)
if choice == 3:
    srt(processes)
if choice == 4:
    rr(processes)
if choice == 5:
    priority_sched(processes)
```